HOMEWORK 3

>>NAME HERE<<
>>ID HERE<<

Instructions:
Use this LaTeX file as a template to develop your homework and submit it as a single PDF file on Gradescope. For the programming component, you can use any programming language, although the teaching staff will be most able to assist with Python and specifically the `sklearn` and `torch` packages; you can use any functionality for Question 3. Put all code used for the assignment at the end of the document in whatever way is easiest but still legible.

1 Short proofs

1.1 Multi-class prediction with the 0-1 loss

Suppose the world generates a single observation $x \sim \text{multinomial}(\theta)$, where the parameter vector $\theta = (\theta_1, \ldots, \theta_k)$ with $\theta_i \geq 0$ and $\sum_{i=1}^k \theta_i = 1$. Note $x \in \{1, \ldots, k\}$. You know θ and want to predict x. Call your prediction $\hat{x}$. What is your expected 0-1 loss:

$$\mathbb{E}1_{\hat{x} \neq x}$$

using the following two prediction strategies respectively? Prove your answer.

1.1.1 Strategy 1 [4 points]

$\hat{x} \in \arg\max_x \theta_x$, the outcome with the highest probability.
Let $i^* = \arg\max_i \theta_i$ and predict $\hat{x} = i^*$. Then

$$\mathbb{E}\,\mathbf{1}_{x \neq \hat{x}} = \Pr(x \neq i^*) = 1 - \Pr(x = i^*) = 1 - \theta_{i^*} = 1 - \max_i \theta_i.$$

1.1.2 Strategy 2 [6 points]

You mimic the world by generating a prediction $\hat{x} \sim \text{multinomial}(\theta)$. (Hint: your randomness and the world's randomness are independent)
Draw $\hat{x} \sim \text{Multinomial}(\theta)$ *independently* of x. Then

$$\Pr(\hat{x} = x) = \sum_{i=1}^k \Pr(x = i) \Pr(\hat{x} = i) = \sum_{i=1}^k \theta_i^2, \quad \Rightarrow \quad \mathbb{E}\,\mathbf{1}_{x \neq \hat{x}} = 1 - \sum_{i=1}^k \theta_i^2.$$

Moreover, Strategy 1 is never worse since $\sum_i \theta_i^2 \leq \max_i \theta_i \cdot \sum_i \theta_i = \max_i \theta_i \Rightarrow 1 - \max_i \theta_i \leq 1 - \sum_i \theta_i^2$.

1.2 Predicting optimally under a weighted multi-class loss [8 points]

Like in the previous question, the world generates a single observation $x \sim \text{multinomial}(\theta)$. Let $c_{ij} \geq 0$ denote the loss you incur, if $x = i$ but you predict $\hat{x} = j$, for $i, j \in \{1, \ldots, k\}$. $c_{ii} = 0$ for all i. This is a way to generalize different costs on false positives vs. false negatives from binary classification to multi-class classification. You want to minimize your expected loss:

$$\mathbb{E}c_{x\hat{x}}.$$

Derive the optimal prediction $\hat{x}$.

Predict class j and incur cost c_{ij} if the truth is i (with $c_{ii} = 0$). For a fixed prediction j the expected loss is

$$L(j) = \mathbb{E}[c_{xj}] = \sum_{i=1}^{k} \theta_i \, c_{ij}.$$

Therefore the Bayes-optimal decision is

$$\hat{x} \;=\; \arg \min_{j \in \{1,\ldots,k\}} \sum_{i=1}^{k} \theta_i \, c_{ij}.$$

(When $c_{ij} = \mathbf{1}\{i \neq j\}$ this reduces to predicting $\arg\max_i \theta_i$.)

1.3 Brute-forcing k-means [7 points]

Consider the following simple brute-force algorithm for minimizing the k-means objective: enumerate all the possible partitionings of the n points into k clusters. For each possible partitioning, compute the optimal centers $c_1, \ldots, c_k$ by taking the mean of the points in each cluster and compute the corresponding k-means objective value. Output the best clustering found. This algorithm is guaranteed to output the optimal set of centers. For the case $k = 2$, argue that the running time of this brute-force algorithm is exponential in the number of data points n. The algorithm enumerates *all* partitions of n points into k nonempty clusters, computes the centers (cluster means) and the k-means objective for each, and returns the best. Even for $k = 2$ the number of unordered two-way nonempty partitions is

$$\frac{2^n - 2}{2} \;=\; 2^{\,n-1} - 1,$$

which is exponential in n. Since evaluating the objective for a *given* partition is only polynomial, the total running time is dominated by the number of partitions and is thus exponential in n.

2 Convolutional neural networks

In this question, you will apply convolutions to the following input:

3	7	2	5
6	4	9	3
1	8	5	6
9	2	3	1

2.1 A basic 2D convolution [5 points]

What is the output when a convolution with the following 2×2 filter is applied to the above input with stride 2, dilation 1, and no padding?

0.1	0.5
0.2	0.3

With stride 2, dilation 1, no padding, the 4×4 input with a 2×2 filter yields a 2×2 output. Using cross-correlation (no kernel flip):

$$\begin{bmatrix} 3 & 7 & 2 & 5 \\ 6 & 4 & 9 & 3 \\ 1 & 8 & 5 & 6 \\ 9 & 2 & 3 & 1 \end{bmatrix} * \begin{bmatrix} 0.1 & 0.5 \\ 0.2 & 0.3 \end{bmatrix} = \begin{bmatrix} 6.2 & 5.4 \\ 6.5 & 4.4 \end{bmatrix}.$$

2.2 Dilated convolutions [5 points]

What is the output when the same filter is applied to the same input but with stride 1, dilation 2, and no padding?
Dilation 2 makes the effective receptive field 3×3 with weights at offsets $(0,0), (0,2), (2,0), (2,2)$. With stride 1, no padding, the 2×2 output is:

$$\begin{bmatrix} 3 \cdot 0.1 + 2 \cdot 0.5 + 1 \cdot 0.2 + 5 \cdot 0.3 & 7 \cdot 0.1 + 5 \cdot 0.5 + 8 \cdot 0.2 + 6 \cdot 0.3 \\ 6 \cdot 0.1 + 9 \cdot 0.5 + 9 \cdot 0.2 + 3 \cdot 0.3 & 4 \cdot 0.1 + 3 \cdot 0.5 + 2 \cdot 0.2 + 1 \cdot 0.3 \end{bmatrix} = \begin{bmatrix} 3.0 & 6.6 \\ 7.8 & 2.6 \end{bmatrix}.$$

2.3 Full matrix equivalent [10 points]

Suppose the input is flattened into a 16 dimensional vector as follows: | 3 | 7 | 2 | $\cdots$ | 2 | 3 | 1 |. What is the matrix that would need to be multiplied by this vector such that when the product is reshaped (by unflattening i.e. applying the inverse of the just-described flattening operation) into a 2×2 matrix, the output will be same as that of the dilated convolution from Question 2.2?
Flatten the input in row-major order to $x \in \mathbb{R}^{16}$:

$$x = [x_1, \ldots, x_{16}] = [3, 7, 2, 5,\ 6, 4, 9, 3,\ 1, 8, 5, 6,\ 9, 2, 3, 1].$$

There exists $W \in \mathbb{R}^{4 \times 16}$ such that $y = Wx$ (then unflatten y to 2×2) equals the dilated-conv output above. The matrix is

$$W = \begin{bmatrix} 0.1 & 0 & 0.5 & 0 & 0 & 0 & 0 & 0 & 0.2 & 0 \\ 0.3 & 0 & 0 & 0 & 0 & 0 & & & & \\ 0 & 0.1 & 0 & 0.5 & 0 & 0 & 0 & 0 & 0 & 0.2 \\ 0 & 0.3 & 0 & 0 & 0 & 0 & & & & \\ 0 & 0 & 0 & 0 & 0.1 & 0 & 0.5 & 0 & 0 & 0 \\ 0 & 0 & 0.2 & 0 & 0.3 & 0 & & & & \\ 0 & 0 & 0 & 0 & 0 & 0.1 & 0 & 0.5 & 0 & 0 \\ 0 & 0 & 0 & 0.2 & 0 & 0.3 & & & & \end{bmatrix}.$$

(Each row picks $(x_1, x_3, x_9, x_{11}), (x_2, x_4, x_{10}, x_{12}), (x_5, x_7, x_{13}, x_{15}), (x_6, x_8, x_{14}, x_{16})$ with weights $(0.1, 0.5, 0.2, 0.3)$, matching dilation 2.) Consequently,

$$y = Wx = \begin{bmatrix} 3 \\ 6.6 \\ 7.8 \\ 2.6 \end{bmatrix},$$

which reshapes (row-major) to the 2×2 output from the dilated convolution.

$$W = \begin{bmatrix} 0.1 & 0 & 0.5 & 0 & 0 & 0 & 0 & 0 & 0.2 & 0 & 0.3 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.1 & 0 & 0.5 & 0 & 0 & 0 & 0 & 0 & 0.2 & 0 & 0.3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0.1 & 0 & 0.5 & 0 & 0 & 0 & 0 & 0 & 0.2 & 0 & 0.3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0.1 & 0 & 0.5 & 0 & 0 & 0 & 0 & 0 & 0.2 & 0 & 0.3 \end{bmatrix}$$

Full matrix W corresponding to the dilated convolution operation. Latex couldnt properly display it

2.4 Convolution as regularization [5 points]

Following the previous question, explain how a convolutional layer can be viewed as a regularization of a regular feed-forward layer.

A convolutional layer is a heavily constrained (sparse, weight-shared) linear map compared to a fully connected layer. Local connectivity enforces spatial locality; weight sharing (same kernel across positions) imposes translation equivariance. These architectural constraints dramatically reduce the number of free parameters and the hypothesis space, acting as structural regularization that improves generalization versus an unconstrained dense layer.

3 Neural network training

Consider a two-layer feed-forward network with h hidden units that takes d-dimensional inputs $\mathbf{x} \in \mathbb{R}^d$ and outputs $\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_2\sigma(\mathbf{W}_1\mathbf{x}))$, where $\mathbf{W}_1 \in \mathbb{R}^{h\times d}$ and $\mathbf{W}_2 \in \mathbb{R}^{k\times h}$ are weights and $\sigma : \mathbb{R}^h \mapsto [0,1]^h$ is the sigmoid activation function. Suppose we want to train this network to minimize the negative log-likelihood objective $L(\hat{\mathbf{y}}, \mathbf{y}) = -\mathbf{y}^\top \log f(\mathbf{x})$ given one-hot-encoded labels $\mathbf{y} \in \{0,1\}^k$.

3.1 Gradient expressions [10 points]

Derive the gradient of $L(\hat{\mathbf{y}}, \mathbf{y})$ w.r.t. $\mathbf{W}_1$ and $\mathbf{W}_2$, in terms of $\mathbf{W}_1, \mathbf{W}_2, \mathbf{x}, \mathbf{y}$, and $\hat{\mathbf{y}}$. You may use the following facts about matrix-vector derivatives without proof:

1. $\frac{\partial L(\text{softmax}(\mathbf{z}),\mathbf{y})}{\partial \mathbf{z}} = \text{softmax}(\mathbf{z}) - \mathbf{y}$
2. if $f : \mathbb{R}^m \mapsto \mathbb{R}$ and $\mathbf{u} = \mathbf{W}\mathbf{v}$ for $\mathbf{W} \in \mathbb{R}^{m\times n}$ and $\mathbf{v} \in \mathbb{R}^n$ then $\frac{\partial f}{\partial \mathbf{v}} = \mathbf{W}^\top \frac{\partial f}{\partial \mathbf{u}}$ and $\frac{\partial f}{\partial \mathbf{W}} = \frac{\partial f}{\partial \mathbf{u}}\mathbf{v}^\top$
3. if $f = g(\mathbf{u})$ and $\mathbf{u} = \sigma(\mathbf{v})$ then $\frac{\partial f}{\partial \mathbf{v}} = \frac{\partial f}{\partial \mathbf{u}} \odot \sigma'(\mathbf{v}) = \frac{\partial f}{\partial \mathbf{u}} \odot \sigma(\mathbf{v}) \odot (\mathbf{1} - \sigma(\mathbf{v}))$

Your answer here.

3.2 Backpropagation algorithm [10 points]

Write down the quantities that need to be stored in the forward pass and the step-by-step (one matrix or vector operation per step) algorithm for computing the two gradients in the backward pass.
Your answer here.

3.3 MNIST classifier [10 points]

Train and test the above model with $h = 128$ on flattened MNIST images using SGD with learning rate 1E-3 and batch size 1. The data consists of 60K training and 10K test points of labeled images of the numbers 0-9, with pixel values normalized to be between 0 and 1 by dividing by 255 if needed. It is easiest to obtain this data via

1. `from torchvision import datasets, transforms`
2. `tt = transforms.ToTensor()`
3. `train = datasets.MNIST('./data', train=True, transform=tt, download=True)`
4. `test = datasets.MNIST('./data', train=False, transform=tt, download=True)`

Initialize the weights at each layer by setting the entries by sampling i.i.d. from the uniform distribution on $[\pm\frac{1}{\sqrt{k}}]$, where k is the input dimension of that layer. Train for five epochs and report (1) the average loss observed across each epoch, (2) the average training accuracy observed across each epoch, and (3) the test accuracy after training.
Your answer here.

3.4 Normalization [5 points]

Normalize the data by subtracting the mean pixel value and dividing by the standard deviation of the pixel values. Report (1) the mean, (2) the standard deviation, and (3) everything reported from Question 3.3. Briefly comment on the effect of doing normalization.
Your answer here.

3.5 Activation functions [5 points]

Replace the sigmoid activation by a ReLU and report everything reported in Question 3.3. Briefly comment on the effect of doing switching to a different activation function.
Your answer here.

3.6 Confusion matrix [5 points]

Plot the confusion matrix of the model trained in Question 3.5, making sure to label the individual matrix entries, and report (1) the most mislabeled ground truth class and (2) the class pair causing the most model confusion and which class is being confused for which other one in that case.
Your answer here.

3.7 Data augmentation [5 points]

Would it be reasonable to do data augmentation via flipping the input image along the vertical axis or rotating the image 180 degrees? Explain why or why not.
Your answer here.